Continuamos con la entrega ininterrumpida de la documentación de ingeniería de software para la **Suite Caja Notarial de San Juan**.
Se han redactado y publicado en el panel Studio dos documentos técnicos completos de especificación e implementación:

1. modulo-02-autenticacion-iam-rbac.md (**Módulo 2: Autenticación, Control de Acceso y Gestión de Identidades**)
2. modulo-03-padron-afiliados-registros.md (**Módulo 3: Padrón de Afiliados, Escribanos y Registros Notariales**)

A continuación, se detalla el contenido técnico íntegro de ambos módulos, listo para ser incorporado al panel de Notas mediante la opción *Guardar en nota*.

Documento de Ingeniería de Software — Módulo 2

Autenticación, Control de Acceso y Gestión de Identidades (IAM & RBAC)

Suite Integral Caja Notarial de San JuanVersión: 1.0.0 | **Estado:** Especificación Técnica Detallada**Requerimientos del Pliego:** SEC-01, SEC-02, SEC-03, SEC-04, SEC-05, USR-01, USR-02, USR-03, USR-04 1-3**Stack de Referencia:** Python 3.12+ | FastAPI | PostgreSQL 16+ | SQLAlchemy 2.0 (Async) | Pydantic v2 | JWT | Argon2id / Bcrypt | PyOTP (2FA TOTP) | React + TypeScript

1. Propósito, Alcance y Mitigación de Defectos Críticos

El **Módulo 2 (IAM & RBAC)** define el perímetro de seguridad e identidad de la suite, estableciendo la autenticación unificada para administración de caja, tesorería, auditoría, escribanos matriculados y público 4.

1.1 Mitigación Directa de Hallazgos Previos

1. **Emisión de enlaces bajo dominio público (SEC-04 / USR-03):** Se erradica de forma terminante el fallo observado en sistemas anteriores donde las notificaciones de restablecimiento emitían enlaces a servidores locales (localhost:8080) 1, 5. En la nueva arquitectura, todo enlace se genera a partir de la variable tipada PUBLIC_WEB_URL. El token es criptográficamente seguro (32 bytes de entropía, salting y hash SHA-256), de un solo uso y con expiración máxima de 30 minutos 1, 6.
2. **Prevención de Enumeración de Cuentas (SEC-04):** El endpoint de solicitud de recuperación responde un mensaje idéntico e indistinguible ("Si la cuenta proporcionada se encuentra registrada y activa, se enviará una comunicación con los pasos a seguir") tanto si el usuario existe como si no existe 1, 6.
3. **Control de Acceso Estricto en Servidor (SEC-01):** Se prohíbe taxativamente considerar el ocultamiento de componentes o botones en React como medida de seguridad 1, 6. Cada endpoint de FastAPI valida los alcances del usuario mediante dependencias declarativas (Depends(require_permissions(...))).
4. **Manejo Seguro de Sesiones y Protección contra Fugas (SEC-05):** Tokens JWT de corta duración (15 a 30 minutos) transmitidos en cabecera Authorization: Bearer,

combinados con Refresh Tokens rotativos almacenados exclusivamente en cookies con directivas HttpOnly, Secure y SameSite=Lax 3.

5. **Autenticación Multifactorial (MFA / 2FA - USR-04):** Soporte nativo para Time-based One-Time Password (TOTP, RFC 6238), obligatorio para roles de alta jerarquía (ADMIN, TESORERIA, CAJA) 2, 4.
6. **Auditoría Inmutable de Accesos y Cambios de Credenciales (SEC-03):** Registro detallado de cada intento de acceso (exitoso o fallido), bloqueo preventivo (tras 5 intentos fallidos consecutivos durante 15 minutos), cambios de clave y revocaciones de permisos, con captura de dirección IP real (X-Forwarded-For), User-Agent y marca temporal en UTC 1.

2. Matriz de Roles y Catálogo Granular de Permisos (RBAC)

2.1 Perfiles y Roles Candidatos (SEC-02 1)

Código de Rol,Denominación Funcional,Alcance Operativo
ROLE_ADMIN,Administrador del Sistema,"Gestión técnica integral, usuarios, roles, parámetros del sistema, auditoría forense 4."
ROLE_CAJA,Administración de Caja,"Padrón de escribanos, recepción de DDJJ, consulta de boletas, carga de novedades y parámetros operativos 4."
ROLE_TESORERIA,Tesorería y Cobranzas,"Verificación y conciliación de comprobantes de pago, liquidación de haberes, reportes contables y financieros 4."
ROLE_ESCRIBANO,Escribano / Afiliado,"Portal de autogestión: carga de DDJJ mensuales, descarga de boletas, adjunción de comprobantes, consulta de préstamos y trámites personales 4."
ROLE_AUDITOR,Auditoría y Fiscalización,"Consulta de sólo lectura de registros históricos, DDJJ, contabilidad, parámetros y logs de auditoría sin capacidad de modificación 4."
ROLE_PUBLICO,Usuario Público,"Acceso a contenidos institucionales, noticias y documentos públicos validados a través del CMS institucional 4."

2.2 Catálogo Granular de Permisos Atómicos

```
# Usuarios y Roles
users:read              # Consultar listados y detalles de
usuarios
users:write             # Crear y editar usuarios
users:suspend           # Suspender / reactivar cuentas de
usuarios (USR-02)
roles:manage            # Asignar roles y modificar permisos

# Padrón de Escribanos y Registros
escribanos:read         # Consultar afiliados y registros
notariales (DIR-01)
escribanos:write        # Alta y modificación de escribanos y
matrículas (DIR-02)
escribanos:status       # Modificar estado de matrícula (activo,
suspendido, cesante)

# Declaraciones Juradas y Boletas
```

```
ddjj:read_own           # Consultar declaraciones juradas propias
(portal escribano)
ddjj:create_own         # Presentar nueva declaración jurada
mensual (DDJJ-01)
ddjj:read_all           # Consultar DDJJ de cualquier escribano
(Caja/Tesorería)
ddjj:approve            # Confirmar y asentar liquidación de
protocolos

# Boletas y Comprobantes
boletas:read_own        # Descargar boleta de pago propia
boletas:read_all        # Consultar y emitir boletas globales
comprobantes:upload     # Adjuntar comprobante bancario a boleta
emitida (DDJJ-04)
comprobantes:verify     # Revisar, conciliar y aprobar/rechazar
comprobante (DDJJ-06, Tesorería)

# Parámetros Previsionales
parameters:read         # Consultar catálogo de alícuotas y
aportes vigentes
parameters:write        # Crear o actualizar parámetros y
vigencias (PAR-01, PAR-04)

# Reportes y Auditoría
reports:financial       # Visualizar y exportar reportes de
recaudación y deuda (REP-01, REP-02)
audit:read              # Visualizar registros inmutables de
auditoría (SEC-03)

# CMS y Sitio Público
cms:publish             # Redactar y publicar noticias, páginas y
estados contables (WEB-01, WEB-02)
```

3. Modelo de Datos Relacional (PostgreSQL DDL)

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Tabla de Roles
CREATE TABLE IF NOT EXISTS security_roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
-- Tabla de Permisos Atómicos
CREATE TABLE IF NOT EXISTS security_permissions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(100) UNIQUE NOT NULL,
    module VARCHAR(50) NOT NULL,
    description TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc', now())
);

-- Tabla Puente Roles - Permisos
CREATE TABLE IF NOT EXISTS security_role_permissions (
    role_id UUID NOT NULL REFERENCES security_roles(id) ON DELETE
CASCADE,
    permission_id UUID NOT NULL REFERENCES
security_permissions(id) ON DELETE CASCADE,
    PRIMARY KEY (role_id, permission_id)
);

-- Tabla Central de Usuarios
CREATE TABLE IF NOT EXISTS security_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    cuit VARCHAR(13) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    full_name VARCHAR(200) NOT NULL,
    phone_number VARCHAR(50),

    -- Control de Estado y Acceso (USR-02)
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    is_verified BOOLEAN NOT NULL DEFAULT FALSE,
    suspension_reason TEXT,
    suspended_at TIMESTAMPTZ,
    suspended_by_user_id UUID REFERENCES security_users(id),

    -- Mecanismo de Segundo Factor (2FA TOTP - USR-04)
    totp_enabled BOOLEAN NOT NULL DEFAULT FALSE,
    totp_secret VARCHAR(64),

    -- Control de Intentos Fallidos
    failed_login_attempts INT NOT NULL DEFAULT 0,
    locked_until TIMESTAMPTZ,
    last_login_at TIMESTAMPTZ,
    last_login_ip VARCHAR(45),

    -- Trazabilidad y Auditoría (AuditMixin)
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
```

```
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_user_id UUID REFERENCES security_users(id)
);

CREATE INDEX idx_users_email ON security_users(email);
CREATE INDEX idx_users_cuit ON security_users(cuit);

-- Tabla Puente Usuarios - Roles
CREATE TABLE IF NOT EXISTS security_user_roles (
    user_id UUID NOT NULL REFERENCES security_users(id) ON DELETE
CASCADE,
    role_id UUID NOT NULL REFERENCES security_roles(id) ON DELETE
CASCADE,
    assigned_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    assigned_by_user_id UUID REFERENCES security_users(id),
    PRIMARY KEY (user_id, role_id)
);

-- Tabla de Tokens de Restablecimiento de Contraseña (SEC-04)
CREATE TABLE IF NOT EXISTS security_password_reset_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES security_users(id) ON DELETE
CASCADE,
    token_hash VARCHAR(64) UNIQUE NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_ip VARCHAR(45) NOT NULL
);

CREATE INDEX idx_reset_token_hash ON
security_password_reset_tokens(token_hash);
CREATE INDEX idx_reset_token_user_expires ON
security_password_reset_tokens(user_id, expires_at);

-- Tabla Central de Auditoría Forense e Inmutable (SEC-03)
CREATE TABLE IF NOT EXISTS audit_security_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_timestamp TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    event_type VARCHAR(100) NOT NULL,
    severity VARCHAR(20) NOT NULL DEFAULT 'INFO',
    actor_user_id UUID REFERENCES security_users(id),
    target_user_id UUID REFERENCES security_users(id),
    ip_address VARCHAR(45) NOT NULL,
```

```
    user_agent TEXT,
    correlation_id VARCHAR(100),
    details JSONB
);

CREATE INDEX idx_audit_event_timestamp ON
audit_security_logs(event_timestamp);
CREATE INDEX idx_audit_actor ON
audit_security_logs(actor_user_id);
CREATE INDEX idx_audit_event_type ON
audit_security_logs(event_type);
```

4. Endpoints de la API REST (FastAPI) e Inyectores de Dependencia

4.1 Inyector de Dependencias de Autorización (backend/app/api/deps.py)

```
import uuid
from typing import List, Callable
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.core.config import settings
from app.core.database import get_db_session
from app.models.security import User

oauth2_scheme =
OAuth2PasswordBearer(tokenUrl=f"{settings.API_V1_STR}/auth/login")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db_session)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="No se pudieron validar las credenciales de
acceso.",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, settings.AUTH_SECRET_KEY,
algorithms=[settings.ALGORITHM])
        user_id_str: str = payload.get("sub")
        if user_id_str is None:
            raise credentials_exception
        user_id = uuid.UUID(user_id_str)
    except (JWTError, ValueError):
        raise credentials_exception
```

```
    stmt = select(User).where(User.id == user_id, User.is_active
== True)
    result = await db.execute(stmt)
    user = result.scalar_one_or_none()
    if user is None:
        raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Usuario inexistente o inactivo.")
    return user

def require_permissions(*required_permissions: str) -> Callable:
    """Inyector de dependencia para validación estricta de
permisos en servidor (SEC-01)."""
    async def permission_checker(
        token: str = Depends(oauth2_scheme),
        current_user: User = Depends(get_current_user)
    ):
        try:
            payload = jwt.decode(token, settings.AUTH_SECRET_KEY,
algorithms=[settings.ALGORITHM])
            user_permissions: List[str] =
payload.get("permissions", [])
            for perm in required_permissions:
                if perm not in user_permissions:
                    raise HTTPException(
                        status_code=status.HTTP_403_FORBIDDEN,
                        detail=f"Permiso insuficiente en servidor.
Se requiere '{perm}' (SEC-01)."
                    )
        except JWTError:
            raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Token no válido.")
        return current_user
    return permission_checker
```

4.2 Controlador de Autenticación y Restablecimiento (backend/app/api/v1/endpoints/auth.py)

```
from datetime import datetime, timedelta, timezone
from fastapi import APIRouter, Depends, HTTPException, Request,
Response, status
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.core.config import settings
from app.core.database import get_db_session
from app.core.security import (
    verify_password, get_password_hash, create_access_token,
```

```
    generate_secure_url_token, hash_token, verify_totp_code
)
from app.models.security import User, PasswordResetToken,
AuditSecurityLog
from app.schemas.security import (
    LoginRequest, TokenResponse, PasswordResetRequest,
PasswordResetConfirm
)

router = APIRouter(prefix="/auth", tags=["Autenticación y
Seguridad"])

@router.post("/login", response_model=TokenResponse)
async def login(
    req: LoginRequest,
    request: Request,
    response: Response,
    db: AsyncSession = Depends(get_db_session)
):
    ip_address = request.client.host if request.client else
"unknown"
    user_agent = request.headers.get("User-Agent", "unknown")
    correlation_id = getattr(request.state, "correlation_id",
None)

    clean_identifier = req.username.strip().replace("-", "")
    stmt = select(User).where(
        (User.email == req.username.strip().lower()) | (User.cuit
== clean_identifier)
    )
    result = await db.execute(stmt)
    user = result.scalar_one_or_none()

    if not user:
        audit = AuditSecurityLog(
            event_type="LOGIN_FAILED_NOT_FOUND",
            severity="WARNING",
            ip_address=ip_address,
            user_agent=user_agent,
            correlation_id=correlation_id,
            details={"identifier": req.username}
        )
        db.add(audit)
        await db.commit()
        raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Credenciales inválidas.")
```

```
    now = datetime.now(timezone.utc)
    if user.locked_until and user.locked_until > now:
        remaining_minutes = int((user.locked_until -
now).total_seconds() / 60)
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail=f"Cuenta bloqueada temporalmente por intentos
fallidos. Reintente en {remaining_minutes} minutos."
        )

    if not verify_password(req.password, user.hashed_password):
        user.failed_login_attempts += 1
        if user.failed_login_attempts >= 5:
            user.locked_until = now + timedelta(minutes=15)

        audit = AuditSecurityLog(
            event_type="LOGIN_FAILED_BAD_PASSWORD",
            severity="WARNING",
            actor_user_id=user.id,
            ip_address=ip_address,
            user_agent=user_agent,
            correlation_id=correlation_id,
            details={"failed_attempts":
user.failed_login_attempts}
        )
        db.add(audit)
        await db.commit()
        raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Credenciales inválidas.")

    if user.totp_enabled:
        if not req.totp_code or not
verify_totp_code(user.totp_secret, req.totp_code):
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Código de autenticación 2FA requerido o
inválido."
            )

    user.failed_login_attempts = 0
    user.locked_until = None
    user.last_login_at = now
    user.last_login_ip = ip_address

    role_codes = [role.code for role in user.roles if
role.is_active]
    permission_set = set()
```

```
    for role in user.roles:
        if role.is_active:
            for perm in role.permissions:
                permission_set.add(perm.code)
    permissions_list = list(permission_set)

    access_token = create_access_token(
        subject=str(user.id),
        permissions=permissions_list,
        roles=role_codes
    )

    audit = AuditSecurityLog(
        event_type="LOGIN_SUCCESS",
        severity="INFO",
        actor_user_id=user.id,
        ip_address=ip_address,
        user_agent=user_agent,
        correlation_id=correlation_id,
        details={"roles": role_codes}
    )
    db.add(audit)
    await db.commit()

    return TokenResponse(
        access_token=access_token,
        expires_in_seconds=settings.ACCESS_TOKEN_EXPIRE_MINUTES *
60,
        user_id=user.id,
        full_name=user.full_name,
        roles=role_codes,
        permissions=permissions_list,
        totp_required=user.totp_enabled
    )

@router.post("/password-reset/request",
status_code=status.HTTP_200_OK)
async def request_password_reset(
    req: PasswordResetRequest,
    request: Request,
    db: AsyncSession = Depends(get_db_session)
):
    ip_address = request.client.host if request.client else
"unknown"
    clean_identifier = req.identifier.strip().replace("-", "")

    stmt = select(User).where(
```

```
        (User.email == req.identifier.strip().lower()) |
(User.cuit == clean_identifier)
    )
    result = await db.execute(stmt)
    user = result.scalar_one_or_none()

    if user and user.is_active:
        raw_token = generate_secure_url_token(32)
        token_hashed = hash_token(raw_token)
        expires_at = datetime.now(timezone.utc) +
timedelta(minutes=settings.PASSWORD_RESET_TOKEN_EXPIRE_MINUTES)

        reset_entry = PasswordResetToken(
            user_id=user.id,
            token_hash=token_hashed,
            expires_at=expires_at,
            created_by_ip=ip_address
        )
        db.add(reset_entry)

        audit = AuditSecurityLog(
            event_type="PASSWORD_RESET_REQUESTED",
            severity="INFO",
            target_user_id=user.id,
            ip_address=ip_address,
            correlation_id=getattr(request.state,
"correlation_id", None),
            details={"expires_at": expires_at.isoformat(),
"public_domain": str(settings.PUBLIC_WEB_URL)}
        )
        db.add(audit)
        await db.commit()

    return {
        "status": "success",
        "message": "Si la cuenta proporcionada se encuentra
registrada y activa, se ha enviado un correo con las instrucciones
de recuperación."
    }

@router.post("/password-reset/confirm",
status_code=status.HTTP_200_OK)
async def confirm_password_reset(
    req: PasswordResetConfirm,
    request: Request,
    db: AsyncSession = Depends(get_db_session)
):
```

```
    ip_address = request.client.host if request.client else
"unknown"
    token_hashed = hash_token(req.token)
    now = datetime.now(timezone.utc)

    stmt = select(PasswordResetToken).where(
        PasswordResetToken.token_hash == token_hashed,
        PasswordResetToken.used_at.is_(None),
        PasswordResetToken.expires_at > now
    )
    result = await db.execute(stmt)
    token_record = result.scalar_one_or_none()

    if not token_record:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="El token de recuperación es inválido, ha
expirado o ya ha sido utilizado (SEC-04)."
        )

    stmt_user = select(User).where(User.id ==
token_record.user_id)
    res_user = await db.execute(stmt_user)
    user = res_user.scalar_one()

    user.hashed_password = get_password_hash(req.new_password)
    user.failed_login_attempts = 0
    user.locked_until = None
    token_record.used_at = now

    audit = AuditSecurityLog(
        event_type="PASSWORD_RESET_COMPLETED",
        severity="INFO",
        actor_user_id=user.id,
        target_user_id=user.id,
        ip_address=ip_address,
        correlation_id=getattr(request.state, "correlation_id",
None),
        details={"status": "password_updated_successfully"}
    )
    db.add(audit)
    await db.commit()

    return {
        "status": "success",
        "message": "La contraseña ha sido actualizada
exitosamente. Ya puede iniciar sesión con sus nuevas
credenciales."
```

```
    }
```

Documento de Ingeniería de Software — Módulo 3

Padrón de Afiliados, Escribanos y Registros Notariales

Suite Integral Caja Notarial de San JuanVersión: 1.0.0 | **Estado:** Especificación Técnica Detallada**Requerimientos del Pliego:** DIR-01, DIR-02, DIR-03, DIR-04, USR-01, USR-02, SEC-01, SEC-03 1, 2, 7**Stack de Referencia:** Python 3.12+ | FastAPI | PostgreSQL 16+ | SQLAlchemy 2.0 (Async) | Pydantic v2 | React + TypeScript

1. Propósito, Alcance y Reglas del Dominio Notarial

El **Módulo 3 (Padrón de Afiliados, Escribanos y Registros)** gestiona el universo de sujetos y entidades habilitadas para el ejercicio notarial en la Provincia de San Juan. Actúa como el maestro de datos de referencia para la posterior emisión de Declaraciones Juradas (DDJJ), cálculo de aportes, emisión de boletas y gestión de préstamos previsionales 7, 8.

1.1 Reglas del Negocio Notarial y Mitigación de Defectos

- **Unicidad e Identidad sin Duplicación (DIR-03 / USR-01 2, 7):** Se erradica la inconsistencia de registrar datos de personas en múltiples tablas desarticuladas. La arquitectura desacopla a la persona física (padron_personas), su matrícula notarial (padron_escribanos) y sus credenciales de acceso al sistema (security_users). El número de CUIT y la matrícula profesional son claves únicas irrepetibles validadas con el algoritmo Módulo 11 oficial de AFIP.
- **Jerarquía y Relación Notarial en San Juan:**
- Cada **Registro Notarial** posee un número identificador único asignado dentro de su circunscripción o departamento territorial de la provincia.
- En cada registro existe exactamente **un Escribano Titular** activo a la vez, pudiendo coexistir uno o más **Escribanos Adscriptos** o Suplentes debidamente designados.
- **Preservación Histórica sin Borrado Físico (DIR-04 2):** Ningún registro de escribano, matrícula o asignación a registro notarial puede ser eliminado de la base de datos mediante sentencias DELETE. Toda baja se implementa como una transición de estado (ACTIVO -> SUSPENDIDO, CESANTE o JUBILADO) con indicación obligatoria del motivo, fecha efectiva y usuario operador (DIR-02 7).
- **Búsqueda Avanzada y Rendimiento (DIR-01 7):** Consultas indexadas por matrícula, CUIT, nombre y apellido, departamento y estado, garantizando respuestas sub-segundo con paginación optimizada.

2. Modelo de Datos Relacional (PostgreSQL DDL)

```
-- Tabla Central de Personas Físicas (Datos Biográficos y
Fiscales)
CREATE TABLE IF NOT EXISTS padron_personas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    cuit VARCHAR(11) UNIQUE NOT NULL, -- 11 dígitos numéricos
limpios
```

```
    tipo_documento VARCHAR(10) NOT NULL DEFAULT 'DNI',
    numero_documento VARCHAR(15) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE,
    genero VARCHAR(20),
    email_personal VARCHAR(255),
    telefono_personal VARCHAR(50),
    domicilio_real TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_user_id UUID REFERENCES security_users(id),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_personas_cuit ON padron_personas(cuit);
CREATE INDEX idx_personas_apellido_nombre ON
padron_personas(apellido, nombre);

-- Tabla de Registros Notariales (Oficinas / Escribanías)
CREATE TABLE IF NOT EXISTS padron_registros_notariales (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    numero_registro INT UNIQUE NOT NULL, -- Número de Registro
Notarial
    nombre_fantasia VARCHAR(150),
    circunscripcion VARCHAR(100) NOT NULL DEFAULT 'Primera
Circunscripción - Capital',
    departamento VARCHAR(100) NOT NULL DEFAULT 'Capital',
    domicilio_legal TEXT NOT NULL,
    telefono_oficina VARCHAR(50),
    email_oficina VARCHAR(255),
    fecha_creacion DATE,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO', -- ACTIVO,
VACANTE, CERRADO
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_user_id UUID REFERENCES security_users(id),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_registros_numero ON
padron_registros_notariales(numero_registro);

-- Tabla de Escribanos Matriculados
```

```
CREATE TABLE IF NOT EXISTS padron_escribanos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    persona_id UUID UNIQUE NOT NULL REFERENCES padron_personas(id)
ON DELETE RESTRICT,
    user_id UUID UNIQUE REFERENCES security_users(id) ON DELETE
SET NULL, -- Cuenta de Autogestión
    matricula_numero INT UNIQUE NOT NULL,
    tomo VARCHAR(20),
    folio VARCHAR(20),
    fecha_matriculacion DATE NOT NULL,
    estado_matricula VARCHAR(30) NOT NULL DEFAULT 'ACTIVO', --
ACTIVO, SUSPENDIDO, CESANTE, JUBILADO, FALLECIDO
    motivo_ultimo_cambio_estado TEXT,
    fecha_ultimo_cambio_estado TIMESTAMPTZ,
    operador_ultimo_cambio_id UUID REFERENCES security_users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_user_id UUID REFERENCES security_users(id),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_escribanos_matricula ON
padron_escribanos(matricula_numero);
CREATE INDEX idx_escribanos_estado ON
padron_escribanos(estado_matricula);

-- Tabla de Asignaciones Notariales (Titularidades y
Adscripciones)
CREATE TABLE IF NOT EXISTS padron_adscripciones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    escribano_id UUID NOT NULL REFERENCES padron_escribanos(id) ON
DELETE RESTRICT,
    registro_id UUID NOT NULL REFERENCES
padron_registros_notariales(id) ON DELETE RESTRICT,
    cargo VARCHAR(30) NOT NULL DEFAULT 'ADSCRIPTO', -- TITULAR,
ADSCRIPTO, SUPLENTE_INTERINO
    resolucion_designacion VARCHAR(100),
    fecha_desde DATE NOT NULL,
    fecha_hasta DATE,
    is_current BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    created_by_user_id UUID REFERENCES security_users(id)
);
```

```
CREATE INDEX idx_adscripciones_escribano ON
padron_adscripciones(escribano_id);
CREATE INDEX idx_adscripciones_registro ON
padron_adscripciones(registro_id);

-- Tabla de Historial Inmutable de Cambios de Estado del Escribano
(DIR-02 / SEC-03)
CREATE TABLE IF NOT EXISTS padron_historial_estados (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    escribano_id UUID NOT NULL REFERENCES padron_escribanos(id) ON
DELETE RESTRICT,
    estado_anterior VARCHAR(30) NOT NULL,
    estado_nuevo VARCHAR(30) NOT NULL,
    motivo TEXT NOT NULL,
    resolucion_respaldo VARCHAR(100),
    fecha_cambio TIMESTAMPTZ NOT NULL DEFAULT timezone('utc',
now()),
    operador_user_id UUID NOT NULL REFERENCES security_users(id)
);

CREATE INDEX idx_historial_escribano ON
padron_historial_estados(escribano_id);
```

3. Validación de CUIT y Contratos Pydantic v2 (backend/app/schemas/padron.py)

```
import re
import uuid
from datetime import date, datetime
from typing import Optional, List
from pydantic import BaseModel, Field, field_validator

def validate_cuit_modulo11(cuit: str) -> str:
    """
    Valida y normaliza un CUIT/CUIL argentino conforme al
algoritmo oficial de Módulo 11.
    Retorna los 11 dígitos limpios si es válido; de lo contrario
lanza ValueError.
    """
    clean_cuit = re.sub(r"\D", "", cuit)
    if len(clean_cuit) != 11:
        raise ValueError("El CUIT debe contener exactamente 11
dígitos numéricos.")

    base = [9-14]
    suma = sum(int(clean_cuit[i]) * base[i] for i in range(10))
    resto = suma % 11
```

```
    if resto == 0:
        digito_verificador = 0
    elif resto == 1:
        if clean_cuit[:2] == "20":
            digito_verificador = 9
        elif clean_cuit[:2] == "27":
            digito_verificador = 4
        else:
            digito_verificador = 0
    else:
        digito_verificador = 11 - resto

    if int(clean_cuit[15]) != digito_verificador:
        raise ValueError("Dígito verificador de CUIT inválido
según algoritmo oficial AFIP.")

    return clean_cuit

class PersonaCreate(BaseModel):
    cuit: str = Field(..., description="CUIT/CUIL de 11 dígitos")
    numero_documento: str = Field(..., min_length=6,
max_length=15)
    tipo_documento: str = "DNI"
    apellido: str = Field(..., min_length=2, max_length=100)
    nombre: str = Field(..., min_length=2, max_length=100)
    fecha_nacimiento: Optional[date] = None
    genero: Optional[str] = None
    email_personal: Optional[str] = None
    telefono_personal: Optional[str] = None
    domicilio_real: Optional[str] = None

    @field_validator("cuit")
    @classmethod
    def check_cuit(cls, v: str) -> str:
        return validate_cuit_modulo11(v)

class EscribanoCreate(BaseModel):
    persona: PersonaCreate
    matricula_numero: int = Field(..., gt=0, description="Número
de matrícula profesional único")
    tomo: Optional[str] = None
    folio: Optional[str] = None
    fecha_matriculacion: date
    registro_id: Optional[uuid.UUID] = Field(None,
description="Registro notarial de asignación inicial")
    cargo_inicial: Optional[str] = Field("ADSCRIPTO",
description="TITULAR o ADSCRIPTO")
```

```
class EscribanoStatusUpdate(BaseModel):
    nuevo_estado: str = Field(..., description="ACTIVO,
SUSPENDIDO, CESANTE o JUBILADO")
    motivo: str = Field(..., min_length=5, description="Motivo
obligatorio del cambio de estado (DIR-02)")
    resolucion_respaldo: Optional[str] = Field(None,
description="Número de resolución de Directorio")

class EscribanoResponse(BaseModel):
    id: uuid.UUID
    matricula_numero: int
    tomo: Optional[str]
    folio: Optional[str]
    fecha_matriculacion: date
    estado_matricula: str
    nombre_completo: str
    cuit: str
    numero_documento: str
    registro_actual: Optional[int] = None
    cargo_actual: Optional[str] = None
    is_active: bool
```

4. Controlador API REST (backend/app/api/v1/endpoints/padron.py)

```
import uuid
from typing import Optional, List
from fastapi import APIRouter, Depends, HTTPException, Query,
status
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func, or_
from app.core.database import get_db_session
from app.api.deps import require_permissions, get_current_user
from app.models.security import User
from app.models.padron import Persona, Escribano,
RegistroNotarial, Adscripcion, HistorialEstado
from app.schemas.padron import EscribanoCreate,
EscribanoStatusUpdate, EscribanoResponse

router = APIRouter(prefix="/padron", tags=["Padrón de Escribanos y
Registros"])

@router.get("/escribanos", response_model=dict)
async def list_escribanos(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    q: Optional[str] = Query(None, description="Búsqueda libre:
apellido, nombre, matrícula o CUIT"),
```

```
    estado: Optional[str] = Query(None, description="Filtro por
estado: ACTIVO, SUSPENDIDO, etc."),
    registro_numero: Optional[int] = Query(None,
description="Filtro por Registro Notarial"),
    db: AsyncSession = Depends(get_db_session),
    current_user: User =
Depends(require_permissions("escribanos:read"))
):
    stmt = select(Escribano).join(Persona, Escribano.persona_id ==
Persona.id)

    if q:
        clean_q = q.strip().replace("-", "")
        stmt = stmt.where(
            or_(
                Persona.apellido.ilike(f"%{q}%"),
                Persona.nombre.ilike(f"%{q}%"),
                Persona.cuit.ilike(f"%{clean_q}%"),
                func.cast(Escribano.matricula_numero,
String).ilike(f"%{q}%")
            )
        )

    if estado:
        stmt = stmt.where(Escribano.estado_matricula ==
estado.upper())

    count_stmt = select(func.count()).select_from(stmt.subquery())
    total_count = (await db.execute(count_stmt)).scalar_one()

    offset = (page - 1) * page_size
    stmt = stmt.order_by(Persona.apellido.asc(),
Persona.nombre.asc()).offset(offset).limit(page_size)
    result = await db.execute(stmt)
    escribanos = result.scalars().all()

    items = []
    for esc in escribanos:
        cargo_act = None
        reg_num = None
        for ads in esc.adscripciones:
            if ads.is_current:
                cargo_act = ads.cargo
                reg_num = ads.registro.numero_registro
                break

        items.append(EscribanoResponse(
            id=esc.id,
```

```
            matricula_numero=esc.matricula_numero,
            tomo=esc.tomo,
            folio=esc.folio,
            fecha_matriculacion=esc.fecha_matriculacion,
            estado_matricula=esc.estado_matricula,
            nombre_completo=f"{esc.persona.apellido},
{esc.persona.nombre}",
            cuit=esc.persona.cuit,
            numero_documento=esc.persona.numero_documento,
            registro_actual=reg_num,
            cargo_actual=cargo_act,
            is_active=esc.is_active
        ))

    return {
        "items": items,
        "total": total_count,
        "page": page,
        "page_size": page_size,
        "total_pages": (total_count + page_size - 1) // page_size
    }

@router.post("/escribanos", response_model=EscribanoResponse,
status_code=status.HTTP_201_CREATED)
async def create_escribano(
    payload: EscribanoCreate,
    db: AsyncSession = Depends(get_db_session),
    current_user: User =
Depends(require_permissions("escribanos:write"))
):
    stmt_cuit = select(Persona).where(Persona.cuit ==
payload.persona.cuit)
    if (await db.execute(stmt_cuit)).scalar_one_or_none():
        raise
HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="El
CUIT ya se encuentra registrado.")

    stmt_mat = select(Escribano).where(Escribano.matricula_numero
== payload.matricula_numero)
    if (await db.execute(stmt_mat)).scalar_one_or_none():
        raise
HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="El
número de matrícula ya existe.")

    persona = Persona(
        cuit=payload.persona.cuit,
        numero_documento=payload.persona.numero_documento,
        tipo_documento=payload.persona.tipo_documento,
```

```
        apellido=payload.persona.apellido,
        nombre=payload.persona.nombre,
        fecha_nacimiento=payload.persona.fecha_nacimiento,
        genero=payload.persona.genero,
        email_personal=payload.persona.email_personal,
        telefono_personal=payload.persona.telefono_personal,
        domicilio_real=payload.persona.domicilio_real,
        created_by_user_id=current_user.id
    )
    db.add(persona)
    await db.flush()

    escribano = Escribano(
        persona_id=persona.id,
        matricula_numero=payload.matricula_numero,
        tomo=payload.tomo,
        folio=payload.folio,
        fecha_matriculacion=payload.fecha_matriculacion,
        estado_matricula="ACTIVO",
        created_by_user_id=current_user.id
    )
    db.add(escribano)
    await db.flush()

    reg_num = None
    if payload.registro_id:
        adscripcion = Adscripcion(
            escribano_id=escribano.id,
            registro_id=payload.registro_id,
            cargo=payload.cargo_inicial or "ADSCRIPTO",
            fecha_desde=payload.fecha_matriculacion,
            is_current=True,
            created_by_user_id=current_user.id
        )
        db.add(adscripcion)

    await db.commit()

    return EscribanoResponse(
        id=escribano.id,
        matricula_numero=escribano.matricula_numero,
        tomo=escribano.tomo,
        folio=escribano.folio,
        fecha_matriculacion=escribano.fecha_matriculacion,
        estado_matricula=escribano.estado_matricula,
        nombre_completo=f"{persona.apellido}, {persona.nombre}",
        cuit=persona.cuit,
        numero_documento=persona.numero_documento,
```

```
        registro_actual=reg_num,
        cargo_actual=payload.cargo_inicial,
        is_active=True
    )

@router.patch("/escribanos/{escribano_id}/status",
status_code=status.HTTP_200_OK)
async def update_escribano_status(
    escribano_id: uuid.UUID,
    payload: EscribanoStatusUpdate,
    db: AsyncSession = Depends(get_db_session),
    current_user: User =
Depends(require_permissions("escribanos:status"))
):
    stmt = select(Escribano).where(Escribano.id == escribano_id)
    escribano = (await db.execute(stmt)).scalar_one_or_none()

    if not escribano:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Escribano no encontrado.")

    estado_anterior = escribano.estado_matricula
    nuevo_estado_norm = payload.nuevo_estado.upper()

    if estado_anterior == nuevo_estado_norm:
        return {"status": "unchanged", "message": "El escribano ya
se encuentra en dicho estado."}

    historial = HistorialEstado(
        escribano_id=escribano.id,
        estado_anterior=estado_anterior,
        estado_nuevo=nuevo_estado_norm,
        motivo=payload.motivo,
        resolucion_respaldo=payload.resolucion_respaldo,
        operador_user_id=current_user.id
    )
    db.add(historial)

    escribano.estado_matricula = nuevo_estado_norm
    escribano.motivo_ultimo_cambio_estado = payload.motivo
    escribano.fecha_ultimo_cambio_estado = func.now()
    escribano.operador_ultimo_cambio_id = current_user.id

    await db.commit()

    return {
        "status": "success",
```

```
        "message": f"Estado de matrícula actualizado de
{estado_anterior} a {nuevo_estado_norm}.",
        "escribano_id": escribano.id
    }
```

Estado del Plan de Trabajo Modular

- x **Módulo 1: Fundamentos de Arquitectura, Entorno y DevOps (Core & Setup)** — Publicado.
- x **Módulo 2: Autenticación, Control de Acceso y Gestión de Identidades (IAM & RBAC)** — Publicado.
- x **Módulo 3: Padrón de Afiliados, Escribanos y Registros Notariales** — Publicado.
- / **Módulo 4: Motor Paramétrico de Aportes y Reglas de Negocio Previsionales** — En preparación.
- **Módulo 5: Declaraciones Juradas (DDJJ), Liquidación de Protocolos y Boletas de Pago**
- **Módulo 6: Gestión de Pagos, Comprobantes y Conciliación de Tesorería**
- **Módulo 7: Reportes Financieros, Historial y Auditoría Transversal**
- **Módulo 8: Sitio Institucional Público y CMS de Estados Contables**
- **Módulo 9: Servicios Previsionales, Préstamos y Beneficios al Afiliado**
- **Módulo 10: Inteligencia Artificial Asistencial y Automatización Notarial**
- **Módulo 11: Estrategia de Migración de Datos, Auditoría Previa y Puesta en Marcha**

💡 **Siguiente paso:** Procedo directamente a redactar y publicar el **Módulo 4: Motor Paramétrico de Aportes y Reglas Previsionales**, estructurando el esquema de vigencias temporales, alícuotas fijas y porcentuales sobre escrituras, tasas de interés por mora e inmutabilidad histórica (PAR-01 a PAR-04 16, 17).